Foundation Models

Machine Learning with Python — Lecture 2, Session 4 (Oct 20)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

A different way to build models — pretrain once, adapt everywhere

- **From task-specific to foundation models** — what changes
- **Text** — embeddings → BERT → GPT and in-context learning
- **Vision** — AlexNet → self-supervised pretraining → ViT
- **Vision–language** — CLIP, multimodal and generative models
- **Why it matters for little data** — fine-tuning, linear probes, zero/few-shot, LoRA and RAG
- **Applications, and the caveats**

Session 4 of 4 today — this wraps the lecture series.

Part 1 — A different way to build models

From task-specific models to foundation models

For most of this course we trained **one model per task**: collect labels, train, deploy. A **foundation model** turns this around:

1. **Pretrain** one huge model on **massive, mostly unlabelled** data (the whole web, billions of images) to learn general structure.
2. **Adapt** it to your task with **little or no labelled data** — fine-tune it, or just **prompt** it.

The pretraining objective is usually **self-supervised** (predict the next word; decide whether two views match), so it needs **no labels at all**. The payoff is **transfer**: a single generalist backbone serves thousands of downstream tasks.

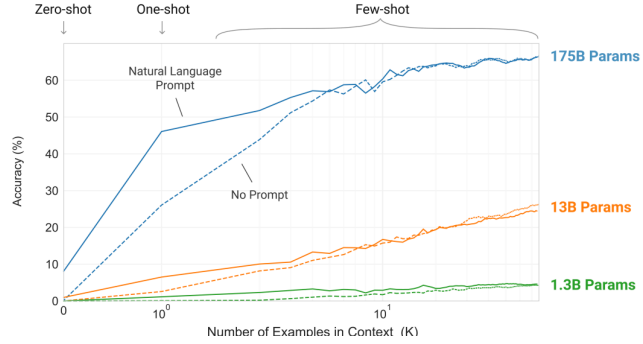
Text — from embeddings to pretraining

- **Word embeddings** (*word2vec*, 2013) learn a vector per word so similar words land close together — *king − man + woman ≈ queen*. But one vector per word cannot handle **context** (*"bank"*).
- **Contextual embeddings** (**BERT**, 2018): a transformer reads the whole sentence, so each word's vector depends on its **context**. BERT is pretrained by **masking** random words and predicting them — pure self-supervision.
- Fine-tune BERT on a task with a **small** labelled set and it beats models trained from scratch on far more data.

Text — generative LLMs

- **GPT** (2018–) is pretrained with a single objective: **predict the next token**. At enough scale, that one objective yields models that summarise, translate, answer questions and write code.
- **Scaling laws** — performance improves smoothly with model size, data and compute; the empirical engine of the last decade.
- **In-context learning** — a large model can perform a *new* task from a few examples **in the prompt**, with **no weight updates** (*few-shot prompting*). **Instruction tuning / RLHF** then align it to be helpful.

Example — in-context learning

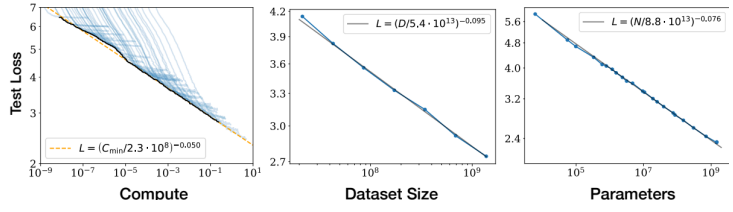


The headline discovery of **GPT-3** (Brown et al., 2020): a large enough language model can perform a **new** task from a few examples placed in the **prompt**, with **no gradient updates** at all.

- a natural-language instruction plus examples raises accuracy;
- the ability **grows with scale** — the 175B model uses the context far better than the 1.3B one;
- "learning" happens at **inference time**, inside the context window.

(Brown et al., Language Models are Few-Shot Learners, 2020 — arXiv:2005.14165)

Example — scaling laws

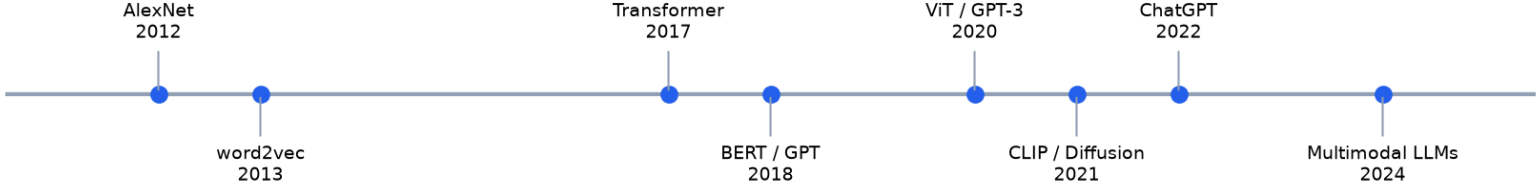


Why does scale work? **Kaplan et al. (2020)** measured test loss across hundreds of runs and found it falls as a smooth **power law** in each of three factors:

- **compute**, **dataset size** and **parameters**;
- the three must grow *together* — scaling one alone quickly saturates;
- this empirical regularity is why the field bets on bigger models and bigger data.

(Kaplan et al., Scaling Laws for Neural Language Models, 2020 — arXiv:2001.08361)

From task-specific models to foundation models



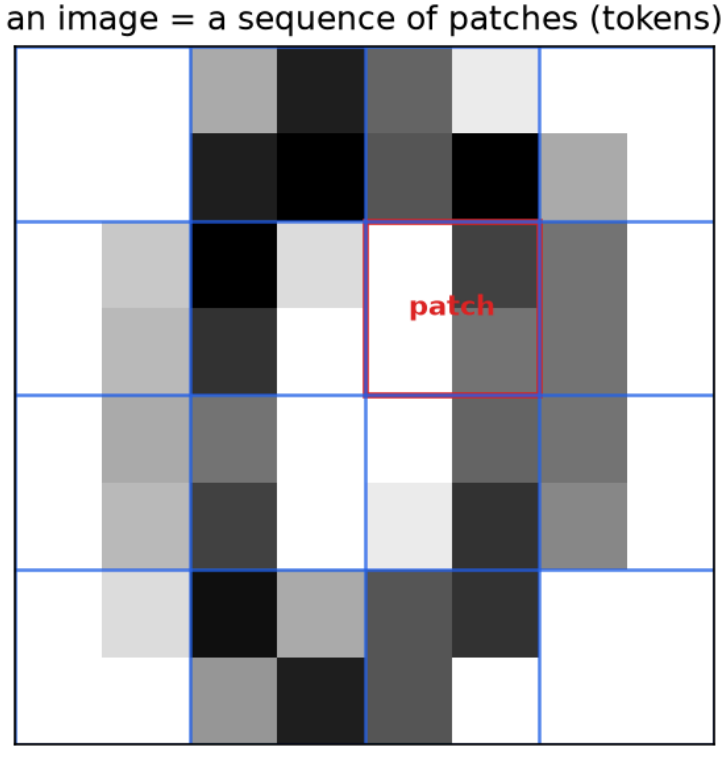
Vision — from supervised to self-supervised

- **AlexNet** (2012) won ImageNet with a deep **CNN** and GPUs — but it was supervised and needed **1.2 million** labelled images.
- The field then asked: can we pretrain **without** labels? Yes:
    - **contrastive learning** (SimCLR, MoCo) — pull two augmentations of the *same* image together, push different images apart;
    - **masked image modelling** (MAE) — hide patches and reconstruct them, the vision analogue of BERT.
- These general visual features **transfer** to detection, segmentation and medical imaging — often with only a handful of labels.

Vision — Vision Transformers (ViT)

The transformer is **not text-specific**. **ViT** (2020) chops an image into **patches** (e.g. 16×16), embeds each patch as a **token**, and runs a plain transformer over them:

- patches play the role of words; **positional encodings** keep their layout;
- with enough pretraining data, ViT matches or beats CNNs — and scales further;
- this **unifies** vision and language under one architecture — the key to the multimodal models next.



Part 2 — Vision and language together

CLIP — connecting images and text

CLIP (2021) trains **two encoders** — one for images, one for text — on **400 million** image–caption pairs:

- for each batch, maximise the similarity of the **correct** (image, caption) pairs and minimise it for every mismatch (**contrastive** loss);
- the result is a **shared embedding space** where an image and its description land in the same place.

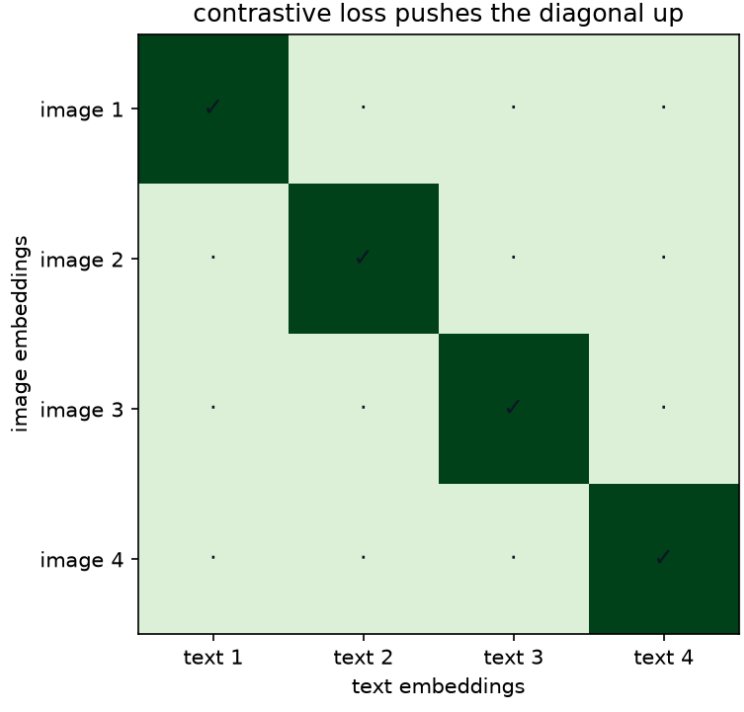
Once trained, CLIP classifies images **zero-shot**: embed the class names ("a photo of a cat"), embed the image, pick the closest — with **no training on the target classes at all**.

The CLIP idea, in one picture

An **image encoder** and a **text encoder** map their inputs into a *shared* space; the training signal is the **similarity matrix**:

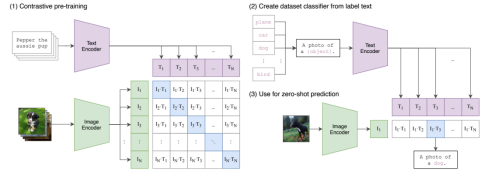
- the **diagonal** (matched image–caption pairs) is pulled together;
- the **off-diagonal** (mismatches) is pushed apart.

Because both live in one space, you can compare an image to *arbitrary text* — the basis for **zero-shot classification**, retrieval and the vision towers inside multimodal LLMs.



Example — CLIP, from the paper

The actual diagram (Radford et al., 2021) — the same three ideas as the previous slide, drawn out:



- **(1) contrastive pre-training** — match N images with N captions in a batch;
- **(2) build a classifier from text** — embed the class names;
- **(3) zero-shot prediction** — pick the caption whose embedding is *closest* to the image. No labelled images for the target task.

(Radford et al., Learning Transferable Visual Models From Natural Language Supervision, 2021 — arXiv:2103.00020)

Multimodal & generative models

- **Vision–language**: captioning, visual question answering, and **multimodal LLMs** (GPT-4V, LLaVA) that both *see* and *talk*.
- **Generative vision**: **diffusion models** (Stable Diffusion, DALL·E) learn to denoise random noise into an image under a text prompt — the same foundation idea, used to **generate** rather than classify.
- **One recipe, many modalities**: audio (Whisper), video, protein sequences, even tabular data — a shared encoder plus a task head, pretrained at scale.

Part 3 — Why this matters for little data

Data efficiency is the whole point

The headline benefit is that foundation models turn a **data** problem into an **adaptation** problem:

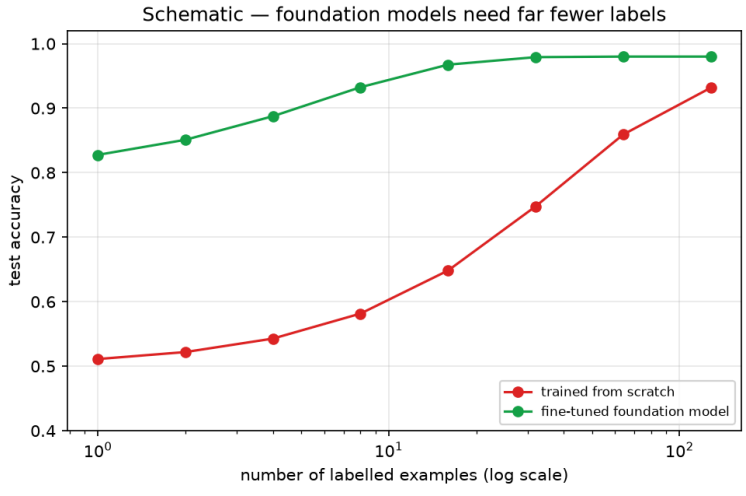
- **Pretrain → fine-tune** — the backbone already "knows" general structure; a little labelled data adapts it to your task.
- **Linear probe** — freeze the backbone and fit a single linear layer on top; often the strongest baseline with **very few** labels.
- **Zero / few-shot** — with CLIP or an LLM you may need **no training at all** — just describe the task.

Few labels, high accuracy

(Illustrative curves — the shape, not the exact numbers, is the point.)

- A model trained **from scratch** needs many labels to climb.
- A pretrained backbone adapted to the task is already good with **a handful** of examples, and improves fast.

This is exactly what makes foundation models usable in the **small-data regimes** that dominate real projects — rare medical conditions, niche industrial defects, small-company datasets.



Adapting cheaply

Fully fine-tuning a billion-parameter model is expensive; a few tricks make adaptation practical:

- **Parameter-efficient fine-tuning (PEFT)** — **LoRA** / adapters *freeze* the backbone and train small injected matrices: fractions of a percent of the parameters.
- **Prompt engineering** — the task lives in the prompt, not the weights.
- **Retrieval-augmented generation (RAG)** — fetch relevant documents and put them in the context, so the model answers from **your** data *without* retraining.

Applications

- **Language** — chatbots, translation, summarisation, code assistants.
- **Vision** — zero-shot classification, segmentation (SAM), detection.
- **Vision–language** — captioning, visual QA, document understanding, medical-imaging reports.
- **Science** — protein structure (AlphaFold), drug discovery, remote sensing.
- **The low-data cases that used to be hopeless** — rare diseases, niche defects on a factory line, small tabular datasets.

Limitations & considerations

- **Compute & cost** — pretraining takes millions of GPU-hours; fine-tuning still needs hardware.
- **Data, licensing & privacy** — scraped data raises consent and copyright questions — and never put private data into a prompt.
- **Bias** — models inherit and can amplify biases in their training data.
- **Hallucination** — fluent text is not the same as fact; verify outputs.
- **Evaluation & reproducibility** — benchmarks saturate and models change; the classical workflow (validation sets, honest baselines) still matters.

Summary

- **Foundation models** are **pretrained** (self-supervised, at scale) and then **adapted** — one generalist backbone, many tasks.
- The arc: **text** (embeddings → BERT/GPT) → **vision** (AlexNet → self-supervised → ViT) → **vision–language** (CLIP) → **multimodal**.
- Their real superpower is **data efficiency** — zero/few-shot, and cheap adaptation via **LoRA**, prompting and **RAG**.
- The training machinery is the same as Session 2 — **backprop + Adam** — just at a scale that changes what is possible.
- Back to reality: for small **labelled** problems, logistic regression and random forests still win on **cost and interpretability**.

That wraps the lecture series — thanks!